Logius Standard - Draft

NLGov REST API Design Rules

Logius Standard
Draft March 04, 2026

This version:
https://logius-standaarden.github.io/API-Design-Rules/

Latest published version:
https://gitdocumentatie.logius.nl/publicatie/api/adr/

Latest editor's draft:
https://logius-standaarden.github.io/API-Design-Rules/

Previous version:
https://gitdocumentatie.logius.nl/publicatie/api/adr/2.0.2/

Editors:

Frank Terpstra (Geonovum)
Jan van Gelder (Geonovum)
Alexander Green (Logius)
Martin van der Plas (Logius)
Tim van der Lippe (Logius)

Authors:

Jasper Roes (Het Kadaster)
Joost Farla (Het Kadaster)

Participate:

GitHub Logius-standaarden/API-Design-Rules
File an issue
Commit history
Pull requests

This document is also available in these non-normative format: PDF

This document is licensed under
Creative Commons Attribution 4.0 International Public License

Status of This Document

This is a draft that could be altered, removed or replaced by other documents. It is not a recommendation approved by TO.

Table of Contents

Status of This Document

Abstract

1. Introduction
1.1 Goal
1.2 Status
1.3 Authors
1.4 Reading Guide
1.5 Extensions

2. The core set of Design Rules
2.1 Summary
2.2 Resources
2.3 HTTP methods
2.4 Statelessness
2.5 Relationships
2.6 Operations
2.7 Documentation
2.8 Versioning
2.9 Transport Security
2.9.1 HTTP-level Security
2.9.2 Browser-based applications
2.9.3 Validate content types
2.10 Normative modules

3. Glossary

A. Linter configuration

B. References
B.1 Normative references
B.2 Informative references

§ Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *NOT RECOMMENDED*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

Abstract

This document contains a normative standard for designing APIs in the Dutch Public Sector. The Governance of this standard is described by the API-Standaarden beheermodel published by Logius. This document is part of the *Nederlandse API Strategie*, which consists of a set of documents.

Date	Version	Changes summary
2020-07-09	1.0	Initial version based on input from Kennisplatform API's
2024-03-07	2.0.0	Add Geospatial and Transport Security modules
2025-08-27	2.1.0	Add `info.contact` rule and linter configuration

§ 1. Introduction

This section is non-normative.

§ 1.1 Goal

More and more governmental organizations offer REST APIs (henceforth abbreviated as APIs), in addition to existing interfaces like SOAP and WFS. These APIs aim to be developer-friendly and easy to implement. While this is a commendable aim, it does not shield a developer from a steep learning curve getting to know every new API, in particular when every individual API is designed using different patterns and conventions.

This document aims to describe a widely applicable set of design rules for the unambiguous provisioning of REST APIs. The primary goal is to offer guidance for organizations designing new APIs, with the purpose of increasing developer experience (DX) and interoperability between APIs. Hopefully, many organizations will adopt these design rules in their corporate API strategies and provide feedback about exceptions and additions to subsequently improve these design rules.

§ 1.2 Status

This version of the design rules has been submitted to Forum Standaardisatie for inclusion on the Comply or Explain list of mandatory standards in the Dutch Public Sector. This document originates from the document API Strategie voor de Nederlandse Overheid, which was recently split into separate sub-documents.

§ 1.3 Authors

Despite the fact that two authors are mentioned in the list of authors, this document is the result of a collaborative effort by the members of the *API Design Rules Working Group*.

§ 1.4 Reading Guide

This document is part of the *Nederlandse API Strategie*.

The Nederlandse API Strategie consists of a set of distinct documents.

Status	Description & Link
Informative	Inleiding NL API Strategie
Informative	Architectuur NL API Strategie
Informative	Gebruikerswensen NL API Strategie
Normative	API Design Rules (ADR v2.0)
Normative	Open API Specification (OAS 3.0)
Normative	NL GOV OAuth profiel
Normative	Digikoppeling REST API koppelvlak specificatie
Normative module	GEO module v1.0
Normative module	Transport Security module v1.0

Before reading this document it is advised to gain knowledge of the informative documents, in particular the Architecture.

An overview of all current documents is available in this Dutch infographic:

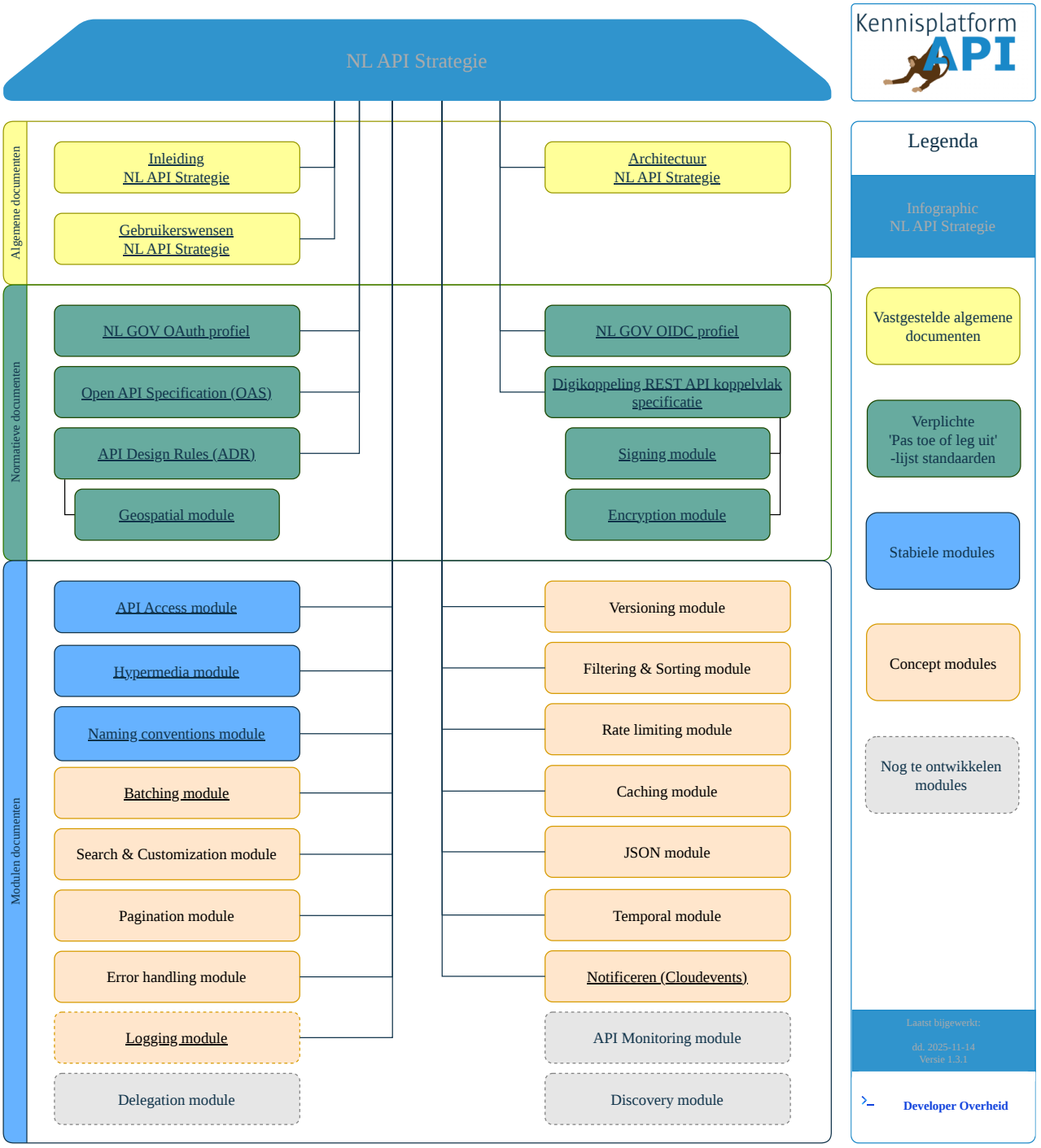


Figure 1 NL API Strategie Infographic

§ 1.5 Extensions

> NOTE
> In addition to this (normative) document, separate modules are being written to provide a set of extensions. These modules are all separate documents and exists in a latest editor's draft (*Werkversie* in Dutch). The latest editor's draft is actively being worked on and can be found on GitHub. It contains the most recent changes.

§ 2. The core set of Design Rules

§ 2.1 Summary

Design rules are technical rules, which should be tested automatically, and functional rules, which should be considered when designing and building the API.

List of technical rules

- /core/no-trailing-slash: Leave off trailing slashes from URIs
- /core/path-segments-kebab-case: Use kebab-case in path segments
- /core/query-keys-camel-case: Use camelCase in query keys
- /core/doc-openapi: Use OpenAPI Specification for documentation
- /core/doc-openapi-contact: Document contact information for publicly available APIs
- /core/publish-openapi: Publish OAS document at a standard location in JSON-format
- /core/uri-version: Include the major version number in the URI
- /core/semver: Adhere to the Semantic Versioning model when releasing API changes
- /core/version-header: Return the full version number in a response header
- /core/transport/tls: Secure connections using TLS
- /core/transport/security-headers: Use mandatory security headers in all API responses
- /core/transport/cors: Use CORS to control access

List of functional rules

- /core/naming-resources: Use nouns to name resources
- /core/naming-collections: Use plural nouns to name collection resources
- /core/interface-language: Define interfaces in Dutch unless there is an official English glossary available
- /core/hide-implementation: Hide irrelevant implementation details
- /core/http-methods: Only apply standard HTTP methods
- /core/http-safety: Adhere to HTTP safety and idempotency semantics for operations
- /core/http-response-code: Adhere to HTTP status codes to convey appropriate errors
- /core/stateless: Do not maintain session state on the server
- /core/nested-child: Use nested URIs for child resources
- /core/resource-operations: Model resource operations as a sub-resource or dedicated resource
- /core/doc-language: Publish documentation in Dutch unless there is existing documentation in English
- /core/deprecation-schedule: Include a deprecation schedule when deprecating features or versions
- /core/transition-period: Schedule a fixed transition period for a new major API version
- /core/changelog: Publish a changelog for API changes between versions
- /core/transport/no-sensitive-uris: No sensitive information in URIs
- /core/modules/geospatial: Apply the geospatial module for geospatial data
- /core/modules/signing: Apply the signing module for signing payloads
- /core/modules/encryption: Apply the encryption module for encrypting payloads

§ 2.2 Resources

The REST architectural style is centered around the concept of a resource. A resource is an abstraction of a conceptual entity, identified by a globally unique URI. It may correspond to anything from a physical object (e.g. a building or a person) to an abstract concept (e.g. a permit, an event or today's weather). Although a resource is not tied to any specific exchange format, its current state can be transferred to clients through one or more representations, such as JSON or XML.

> **/core/naming-resources**: Use nouns to name resources
>
> Functional

Statement

Resources *MUST* be referred to using nouns (instead of verbs) that represent entities meaningful to the API consumer.

EXAMPLE 1

A few correct examples of nouns as part of a URI:

- Gebouw
- Vergunning

This is different than RPC-style APIs, where verbs are often used to perform certain actions:

- Opvragen
- Registreren

Rationale

Resources describe objects, not actions.

A resource that corresponds to a single conceptual entity is referred to as a singular resource. Resources can also be logically grouped into collections, which are themselves resources and typically support operations like paging, sorting, and filtering. While collection members are often of the same type, this is not strictly required. A collection resource contains references (URIs) to the individual singular resources it includes.

Functional

/core/naming-collections: Use plural nouns to name collection resources

Statement

Collection resources *MUST* be referred to using plural nouns.

Rationale

The path segment describing the name of the collection resource *MUST* be written in the plural form.

Singular resources contained within a collection resource are generally named by appending a path segment for the identification of each individual resource.

Singular resources that stand on their own, i.e. which are not contained within a collection resource, *MUST* be named with a path segment that is written in the singular form.

EXAMPLE 2

Collection resources describe a list of things:

```
https://api.example.org/v1/gebouwen
https://api.example.org/v1/vergunningen
```

Singular resource that is contained within a collection resource:

```
https://api.example.org/v1/gebouwen/3b9710c4-6614-467a-ab82
https://api.example.org/v1/vergunningen/d285e05c-6b01-45c3-
```

Singular resource describing the profile of the currently authenticated user:

```
https://api.example.org/v1/gebruikersprofiel
```

Functional

/core/interface-language: Define interfaces in Dutch unless there is an official English glossary available

Statement

Resources and the underlying attributes *SHOULD* be defined in the Dutch language unless there is an official English glossary available.

Rationale

If your API references terms used in law or official government communication for example, then these terms have a well-defined meaning. The exact meaning of concepts is often lost in translation, hence such terms *SHOULD* be defined in the Dutch language.

Publishing an API for an international (e.g. European) audience might be a reason to define interfaces in English instead.

Note that glossaries exist that define useful sets of attributes which *SHOULD* preferably be reused. Examples can be found at schema.org.

Technical

/core/no-trailing-slash: Leave off trailing slashes from URIs

Statement

A URI *MUST NOT* contain a trailing slash. When requesting a resource including a trailing slash, this *MUST* result in a `404` (not found) error response and not a

redirect. This forces API consumers to use the correct URI.

> NOTE
> This rule does not apply to the root resource (append `/` to the service root URL).

Rationale

Leaving off trailing slashes, and not implementing a redirect, forces API consumers to use the correct URI. This avoids confusion and ambiguity.

> EXAMPLE 3
>
> URI without a trailing slash (correct):
>
> `https://api.example.org/v1/gebouwen`
>
> URI with a trailing slash (incorrect):
>
> `https://api.example.org/v1/gebouwen/`
>
> URI for the root resource is exempt (correct):
>
> `https://api.example.org/v1/`

How to test

Analyse all resource paths (except the root resource path) in the OpenAPI Description to confirm no resource paths end with a forward slash (`/`). This rule can be automatically checked and an example test is shown in the linter configuration.

Technical

/core/path-segments-kebab-case: Use kebab-case in path segments

Statement

Path segments of a URI *MUST* only contain lowercase letters, digits or hyphens. This is also known as kebab-case. Hyphens *MUST* only be used to deliniate distinct words. This also implies that diacritics *MUST* be normalized and special characters *MUST* be omitted.

Another implication of this rule is that file extensions *MUST NOT* be used. Resources *SHOULD* use the `Accept` header for content negotation.

The last path segment *MAY* start with **_**, which is used as a convention to implement operations

Rationale

Some web servers and frameworks do not handle case sensitivity or special characters of URIs well. The use of kebab-case path segments ensures compatibility with a broad range of systems. It is a more common implementation choice for path segments than camelCase or snake_case. Information (such as names of objects) that requires special characters can be part of the request body instead of being in the URI.

EXAMPLE 4

URI path segment using kebab-case (correct):

`https://api.example.org/v1/financiele-claims`

URI path segment not using hyphens to delineate words (incorrect):

`https://api.example.org/v1/financiele_claims`

URI path segment not lowercase characters (incorrect):

`https://api.example.org/v1/financieleClaims`

URI path segment ending with a hyphen (incorrect):

`https://api.example.org/v1/organisatie-`

URI path segment starting with a hyphen (incorrect):

`https://api.example.org/v1/-organisatie`

URI path segment using normalized diacritics (correct):

`https://api.example.org/v1/scenes`

URI path segment using diacritics (incorrect):

`https://api.example.org/v1/scènes`

URI path segment omitting special characters (correct):

`https://api.example.org/v1/schemas`

URI path segment using special characters (incorrect):

`https://api.example.org/v1/schema's`

URI path segment using file extensions (incorrect):

`https://api.example.org/v1/schema.txt`

Last URI path segment starting with _ (correct):

`https://api.example.org/v1/organisaties/_zoek`

How to test

Loop all resource paths in the OpenAPI Description and check that all resource path segments use lowercase letters, digits or hyphens (`-`). The last path segment is

allowed to start with a _.

EXAMPLE 5

You can use the following regex for each resource path:

```
^(\/|(\/_[a-z0-9]+|\/(([a-z0-9\-]+|{[^}]+})(\/([a-z0-9\-\.
```

This rule can be automatically checked and an example test is shown in the linter configuration.

Technical

/core/query-keys-camel-case: Use camelCase in query keys

Statement

Query keys in a URI *MUST* only contain letters and digits, where the first letter of each word is capitalized, except for the first letter (*MUST NOT* be a digit) of the entire compound word. This is also known as lower camelCase. This also implies that diacritics *MUST* be normalized and special characters *MUST* be omitted.

Rationale

Query keys are often converted to JSON object keys, where camelCase is the naming convention to avoid compatibility issues with JavaScript when deserializing objects.

EXAMPLE 6

URI query key using camelCase (correct):

```
https://api.example.org/v1/gebouwen?typeGebouw=woning
```

URI query key not using camelCase (incorrect):

```
https://api.example.org/v1/gebouwen?type-gebouw=woning
```

URI query key starts with digit (incorrect):

```
https://api.example.org/v1/gebouwen?2ndReviewer=alice
```

How to test

Loop all resource paths in the OpenAPI Description and check that all query keys use letters, digits in camelCase. You can use the following regex for each query

key:

EXAMPLE 7

```
^\$?[a-z][a-z\d]*([A-Z][a-z\d]*)*$
```

This rule can be automatically checked and an example test is shown in the linter configuration.

Functional

/core/hide-implementation: Hide irrelevant implementation details

Statement

An API *SHOULD NOT* expose implementation details of the underlying application, development platforms/frameworks or database systems/persistence models.

Rationale

- The primary motivation behind this design rule is that an API design *MUST* focus on usability for the client, regardless of the implementation details under the hood.
- The API, application and infrastructure need to be able to evolve independently to ease the task of maintaining backwards compatibility for APIs during an agile development process.
- The API design of Convenience,- and Process API types (as described in Aanbeveling 2 of the NL API Strategie) *SHOULD NOT* be a 1-on-1 mapping of the underlying domain- or persistence model.
- The API design of a System API type (as described in Aanbeveling 2 of the NL API Strategie) *MAY* be a mapping of the underlying persistence model.
- The API *SHOULD NOT* expose information about the technical components being used, such as development platforms/frameworks or database systems.
- The API *SHOULD* offer client-friendly attribute names and values, while persisted data may contain abbreviated terms or serializations which might be cumbersome for consumption.

§ 2.3 HTTP methods

Although the REST architectural style does not impose a specific protocol, REST APIs are typically implemented using HTTP [*RFC9110*].

Functional

/core/http-methods: Only apply standard HTTP methods

Statement

An API *MUST* adhere to the HTTP method semantics defined in [*RFC9110*].

Rationale

The HTTP specifications offer a set of standard methods, where every method is designed with explicit semantics. Adhering to the HTTP specification is crucial, since HTTP clients and middleware applications rely on standardized characteristics.

Method	Operation	Description
`GET`	Read	Retrieve a resource representation for the given URI. Data is only retrieved and never modified.
`POST`	Create	Create a subresource as part of a collection resource. This operation is not relevant for singular resources. This method can also be used for exceptional cases.
`PUT`	Create/update	Create a resource with the given URI or replace (full update) a resource when the resource already exists.
`PATCH`	Update	Partially updates an existing resource. The request only contains the resource modifications instead of the full resource representation.
`DELETE`	Delete	Remove a resource with the given URI.

EXAMPLE 8

The following table shows some examples of the use of standard HTTP methods:

Request	Description
`GET /rijksmonumenten`	Retrieves a list of national monuments.
`GET /rijksmonumenten/12`	Retrieves an individual national monument.
`POST /rijksmonumenten`	Creates a new national monument.
`PUT /rijksmonumenten/12`	Modifies national monument #12 completely.
`PATCH /rijksmonumenten/12`	Modifies national monument #12 partially.
`DELETE /rijksmonumenten/12`	Deletes national monument #12.

NOTE

The HTTP specification [*RFC9110*] offers a set of standard methods, where every method is designed with explicit semantics. HTTP also defines other methods, e.g. `HEAD`, `OPTIONS`, `TRACE`, and `CONNECT`.
The OpenAPI Specification 3.0 Path Item Object also supports these methods, except for `CONNECT`.
According to RFC 9110 9.1 the `GET` and `HEAD` HTTP methods *MUST* be supported by the server, all other methods are optional.
In addition to the standard HTTP methods, a server may support other optional methods as well, e.g. `PROPFIND`, `COPY`, `PURGE`, `VIEW`, `LINK`, `UNLINK`, `LOCK`, `UNLOCK`, etc.
If an optional HTTP request method is sent to a server and the server does not support that HTTP method for the target resource, an HTTP status code `405 Method Not Allowed` shall be returned and a list of allowed methods for the target resource shall be provided in the `Allow` header in the response as stated in RFC 9110 15.5.6.

How to test

Analyse the OpenAPI Description to confirm all supported methods are either `post`, `put`, `get`, `delete`, or `patch`.

Functional

/core/http-safety: Adhere to HTTP safety and idempotency semantics for operations

Statement

The following table describes which HTTP methods *MUST* behave as safe and/or idempotent:

Method	Safe	Idempotent
`GET`	Yes	Yes
`HEAD`	Yes	Yes
`OPTIONS`	Yes	Yes
`POST`	No	No
`PUT`	No	Yes
`PATCH`	No	No
`DELETE`	No	Yes

Rationale

The HTTP protocol [*RFC9110*] specifies whether an HTTP method *SHOULD* be considered safe and/or idempotent. These characteristics are important for clients and middleware applications, because they *SHOULD* be taken into account when implementing caching and fault tolerance strategies.

Request methods are considered *safe* if their defined semantics are essentially read-only; i.e., the client does not request, and does not expect, any state change on the origin server as a result of applying a safe method to a target resource. A request method is considered *idempotent* if the intended effect on the server of multiple identical requests with that method is the same as the effect for a single such request.

Functional

/core/http-response-code: Adhere to HTTP status codes to convey appropriate errors

Statement

An API *MUST* use the semantically appropriate HTTP status code ([*RFC9110*]) for the response.

Rationale

The server *SHOULD NOT* only use `200` for success and `404` for error states. Use the semantically appropriate status code for success or failure.

> In case of an error, the server *SHOULD NOT* pass technical details (e.g. call stacks or other internal hints) to the client. The error message *SHOULD* be generic to avoid revealing additional details and expose internal information which can be used with malicious intent.

§ 2.4 Statelessness

One of the key constraints of the REST architectural style is stateless communication between client and server. It means that every request from client to server must contain all of the information necessary to understand the request. The server cannot take advantage of any stored session context on the server as it didn't memorize previous requests. Session state must therefore reside entirely on the client.

To properly understand this constraint, it is important to make a distinction between two different kinds of state:

- *Session state*: information about the interactions of an end user with a particular client application within the same user session, such as the last page being viewed, the login state or form data in a multi-step registration process. Session state must reside entirely on the client (e.g. in the user's browser).
- *Resource state*: information that is permanently stored on the server beyond the scope of a single user session, such as the user's profile, a product purchase or information about a building. Resource state is persisted on the server and must be exchanged between client and server (in both directions) using representations as part of the request or response payload. This is actually where the term *REpresentational State Transfer (REST)* originates from.

> NOTE
>
> It is a misconception that there should be no state at all. The stateless communication constraint should be seen from the server's point of view and states that the server should not be aware of any *session state*.

Stateless communication offers many advantages, including:

- *Simplicity* is increased because the server does not have to memorize or retrieve session state while processing requests
- *Scalability* is improved because not having to incorporate session state across multiple requests enables higher concurrency and performance

- *Observability* is improved since every request can be monitored or analyzed in isolation without having to incorporate session context from other requests
- *Reliability* is improved because it eases the task of recovering from partial failures since the server does not have to maintain, update or communicate session state. One failing request does not influence other requests (depending on the nature of the failure of course).

Functional

/core/stateless: Do not maintain session state on the server

Statement

In the context of REST APIs, the server *MUST NOT* maintain or require any notion of the functionality of the client application and the corresponding end user interactions.

Rationale

To achieve full decoupling between client and server, and to benefit from the advantages mentioned above, session state *MUST NOT* reside on the server. Session state *MUST* therefore reside entirely on the client.

NOTE

The client of a REST API could be a variety of applications such as a browser application, a mobile or desktop application and even another server serving as a backend component for another client. REST APIs should therefore be completely client-agnostic.

§ 2.5 Relationships

Resources are often interconnected by relationships. Relationships can be modelled in different ways depending on the cardinality, semantics and more importantly, the use cases and access patterns the REST API needs to support.

Functional

/core/nested-child: Use nested URIs for child resources

Statement

When having a child resource which can only exist in the context of a parent resource, the URI *SHOULD* be nested.

Rationale

In this use case, the child resource does not necessarily have a top-level collection resource. The best way to explain this design rule is by example.

EXAMPLE 9

When modelling resources for a news platform including the ability for users to write comments, it might be a good strategy to model the collection resources hierarchically:

```
https://api.example.org/v1/articles/123/comments
```

The platform might also offer a photo section, where the same commenting functionality is offered. In the same way as for articles, the corresponding sub-collection resource might be published at:

```
https://api.example.org/v1/photos/456/comments
```

These nested sub-collection resources can be used to post a new comment (`POST` method) and to retrieve a list of comments (`GET` method) belonging to the parent resource, i.e. the article or photo. An important consideration is that these comments could never have existed without the existence of the parent resource.

From the consumer's perspective, this approach makes logical sense, because the most obvious use case is to show comments below the parent article or photo (e.g. on the same web page) including the possibility to paginate through the comments. The process of posting a comment is separate from the process of publishing a new article. Another client use case might also be to show a global *latest comments* section in the sidebar. For this use case, an additional resource could be provided:

```
https://api.example.org/v1/comments
```

If this would have not been a meaningful use case, this resource should not exist at all. Because it does not make sense to post a new comment from a global context, this resource would be read-only (only `GET` method is supported) and may possibly provide a more compact representation than the parent-specific sub-collections.

The singular resources for comments, referenced from all 3 collections, could still be modelled on a higher level to avoid deep nesting of URIs (which might increase complexity or problems due to the URI length):

```
https://api.example.org/v1/comments/123
https://api.example.org/v1/comments/456
```

Although this approach might seem counterintuitive from a technical perspective (we simply could have modelled a single `/comments` resource with optional filters for article and photo) and might introduce partially

redundant functionality, it makes perfect sense from the perspective of the consumer, which increases developer experience.

§ 2.6 Operations

Functional

/core/resource-operations: Model resource operations as a sub-resource or dedicated resource

Statement

Resource operations *MUST* be modelled as a sub-resource or dedicated resource.

Rationale

There are resource operations which might not seem to fit well in the CRUD interaction model. For example, approving a submission or notifying a customer. Depending on the type of the operation, there are three possible approaches:

1. Re-model the resource to incorporate extra fields supporting the particular operation. For example, an approval operation can be modelled in a boolean attribute `goedgekeurd` that can be modified by issuing a `PATCH` request against the resource. A drawback of this approach is that the resource does not contain any metadata about the operation (when and by whom was the approval given? Was the submission rejected in an earlier stage?). Furthermore, this requires a fine-grained authorization model, since approval might require a specific role.

2. Treat the operation as a sub-resource. For example, model a sub-collection resource `/inzendingen/12/beoordelingen` and add an approval or rejection by issuing a `POST` request. To be able to retrieve the review history (and to consistently adhere to the REST principles), also support the `GET` method for this resource. The `/inzendingen/12` resource might still provide a `goedgekeurd` boolean attribute (same as approach 1) which gets automatically updated in the background after adding a review. This attribute *SHOULD* however be read-only.

3. In exceptional cases, the approaches above still do not offer an appropriate solution. An example of such an operation is a global search across multiple resources. In this case, the creation of a dedicated resource, possibly nested under an existing resource, is the most obvious solution. Use the imperative mood of a verb, maybe even prefix it with a underscore to distinguish these resources from regular resources. For example: `/search` or `/_search`.

Depending on the operation characteristics, `GET` and/or `POST` method *MAY* be supported for such a resource.

§ 2.7 Documentation

An API is as good as the accompanying documentation. The documentation has to be easily findable, searchable and publicly accessible. Most developers will first read the documentation before they start implementing. Hiding the technical documentation in PDF documents and/or behind a login creates a barrier for both developers and search engines.

Technical

/core/doc-openapi: Use OpenAPI Specification for documentation

Statement

API documentation *MUST* be provided in the form of an OpenAPI definition document which conforms to the OpenAPI Specification (from v3 onwards).

Rationale

The OpenAPI Specification (OAS) [*OPENAPIS*] defines a standard, language-agnostic interface to RESTful APIs which allows both humans and computers to discover and understand the capabilities of the service without access to source code, documentation, or through network traffic inspection. When properly defined, a consumer can understand and interact with the remote service with a minimal amount of implementation logic. API documentation *MUST* be provided in the form of an OpenAPI definition document which conforms to the OpenAPI Specification (from v3 onwards). As a result, a variety of tools can be used to render the documentation (e.g. Swagger UI or ReDoc) or automate tasks such as testing or code generation. The OAS document *SHOULD* provide clear descriptions and examples.

How to test

Parse the resource at the provided location as an OpenAPI Description and confirm all $refs are resolvable and paths are defined. This rule can be automatically checked and an example test is shown in the linter configuration.

Technical

/core/doc-openapi-contact: Document contact information for publicly available APIs

Statement

OpenAPI definition document *SHOULD* include the `info.contact` object for publicly available APIs. Contact information *SHOULD NOT* be a generic contact address for the whole organisation.

Rationale

The OpenAPI Specification (OAS) [*OPENAPIS*] can include contact information to make clear how to reach out to API owners in case of issues or questions. This is relevant for publicly available APIs (such as OData) where no pre-existing communication channel exists between provider and consumer of the API. For internal APIs (where communication channels such as chat or issue trackers are likely already known), the `info.contact` *MAY* be provided.

EXAMPLE 10

Relevant contact information can include an email address and issue tracker.

```
{
  "name": "Gebouwen API beheerder",
  "url": "https://www.github.com/ministerie/gebouwen/issue
  "email": "teamgebouwen@ministerie.nl"
}
```

How to test

Parse the OpenAPI Description to confirm the `info.contact` object is present. This rule can be automatically checked and an example test is shown in the linter configuration.

Functional

/core/doc-language: Publish documentation in Dutch unless there is existing documentation in English

Statement

You *SHOULD* write the OAS document in Dutch.

Rationale

In line with design rule /core/interface-language, the OAS document (e.g. descriptions and examples) *SHOULD* be written in Dutch. If relevant, you *MAY* refer to existing documentation written in English.

Technical

/core/publish-openapi: Publish OAS document at a standard location in JSON-format

Statement

To make the OAS document easy to find and to facilitate self-discovering clients, there *SHOULD* be one standard location where the OAS document is available for download.

Rationale

It *MUST* be possible for clients (such as Swagger UI or ReDoc) to retrieve the document without having to authenticate. Furthermore, the CORS policy for this URI *MUST* allow external domains to read the documentation from a browser environment.

The standard location for the OAS document is a URI called `openapi.json` or `openapi.yaml` within the base path of the API. This can be convenient, because OAS document updates can easily become part of the CI/CD process.

At least the JSON format *MUST* be supported. When having multiple (major) versions of an API, every API version *SHOULD* provide its own OAS document(s).

> EXAMPLE 11
> An API having base path `https://api.example.org/v1` *MUST* publish the OAS document at:
>
> `https://api.example.org/v1/openapi.json`
>
> Optionally, the same OAS document *MAY* be provided in YAML format:
>
> `https://api.example.org/v1/openapi.yaml`

How to test

- Step 1: The API *MUST* meet the prerequisites to be tested. These include that an OAS file (openapi.json) is publicly available, parsable, all $refs are resolvable and paths are defined.
- Step 2: The openapi.yaml document *MAY* be available. If available it *MUST* contain YAML, be readable and parsable.
- Step 3: The openapi.yaml document *MUST* contain the same OpenAPI Description as the openapi.json document.

- Step 4: The CORS header Access-Control-Allow-Origin *MUST* allow all origins.

§ 2.8 Versioning

Changes in APIs are inevitable. APIs should therefore always be versioned, facilitating the transition between changes.

Functional

/core/deprecation-schedule: Include a deprecation schedule when deprecating features or versions

Statement

The API owner *SHOULD* implement well-documented deprecation schedules that are communicated in a timely fashion.

Rationale

Managing change is important. In general, good documentation and timely communication regarding deprecation schedules are the most important for API users. When deprecating features or versions, a deprecation schedule *MUST* be published. This document *SHOULD* be published on a public web page. Furthermore, active clients *SHOULD* be informed by e-mail once the schedule has been updated or when versions have reached end-of-life.

Functional

/core/transition-period: Schedule a fixed transition period for a new major API version

Statement

Old versions *MUST* remain available for a limited and fixed deprecation period.

Rationale

When releasing a new major API version, the old version *MUST* remain available for a limited and fixed deprecation period. Offering a deprecation period allows clients to carefully plan and execute the migration from the old to the new API version, as long as they do this prior to the end of the deprecation period. A maximum of 2 major API versions *MAY* be published concurrently.

Technical

/core/uri-version: Include the major version number in the URI

Statement

The URI of an API *MUST* include the major version number.

Rationale

The URI of an API (base path) *MUST* include the major version number, prefixed by the letter `v`. This allows the exploration of multiple versions of an API in the browser. The minor and patch version numbers are not part of the URI and *MAY* not have any impact on existing client implementations.

EXAMPLE 12

An example of an `openapi.yaml` for an API with a base path `https://api.example.org/v1` and current version 1.0.2:

```
openapi: 3.0.0
   info:
      version: '1.0.2'
   servers:
      - description: test environment
      url: https://api.test.example.org/v1
      - description: production environment
      url: https://api.example.org/v1
```

How to test

Parse the `url` field in the `servers` mentioned in the OpenAPI Description to confirm that a version number is present with prefix `v` and only contains the *major* version number. This rule can be automatically checked and an example test is shown in the linter configuration.

Functional

/core/changelog: Publish a changelog for API changes between versions

Statement

A changelog *MUST* be publised for every API version.

Rationale

When releasing new (major, minor or patch) versions, all API changes *MUST* be documented properly in a publicly available changelog.

Technical

/core/semver: Adhere to the Semantic Versioning model when releasing API changes

Statement

Semantic Versioning *MUST* be used for API versioning.

Rationale

Version numbering *MUST* follow the Semantic Versioning [*SemVer*] model to prevent breaking changes when releasing new API versions. Release versions are formatted using the `major.minor.patch` template (examples: 1.0.2, 1.11.0). Pre-release versions *MAY* be denoted by appending a hyphen and a series of dot separated identifiers (examples: 1.0.2-rc.1, 2.0.0-beta.3). When releasing a new version which contains backwards-incompatible changes, a new major version *MUST* be released. Minor and patch releases *MUST* only contain backwards compatible changes (e.g. the addition of an endpoint or an optional attribute).

How to test

Parse the `info.version` field in the OpenAPI Description to confirm it adheres to the Semantic Versioning format.

Technical

/core/version-header: Return the full version number in a response header

Statement

The API-Version header *MUST* be returned in a response header in every response.

Rationale

Since the URI only contains the major version, it is useful to provide the full version number in the response headers for every API call. This information could then be used for logging, debugging or auditing purposes. In cases where an intermediate networking component returns an error response (e.g. a reverse proxy enforcing access policies), the version number *MAY* be omitted.

The version number *MUST* be returned in an HTTP response header named `API-Version` (case-insensitive) and *SHOULD NOT* be prefixed.

EXAMPLE 13

An example of an API version response header:

```
API-Version: 1.0.2
```

How to test

A response includes a header "API-Version" with a number matching the version number set in the `info.version` field of the OpenAPI Description. This rule can be automatically checked and an example test is shown in the linter configuration.

§ 2.9 Transport Security

This section describes security principles, concepts and technologies to apply when working with APIs. Controls need to be applied for the security objectives of integrity, confidentiality and availability of the API (which includes the services and data provided thereby). The architecture section of the API strategy contains architecture patterns for implementing transport security.

The scope of this section is limited to generic security controls that directly influence the visible parts of an API. Effectively, only security standards directly applicable to interactions are discussed here.

In order to meet the complete security objectives, every implementer *MUST* also apply a range of controls not mentioned in this section.

Note: security controls for signing and encrypting of application level messages are part of separate extensions: Signing and Encryption.

Technical

/core/transport/tls: Secure connections using TLS

Statement

Information exchanged by APIs *MUST* be secured using TLS-based connections. No exceptions, so everywhere and always. This is required by law.

One *MUST* follow the latest NCSC guidelines [*NCSC 2025*].

Rationale

Since the connection is always secured, the access method can be straightforward. This allows the application of basic access tokens instead of encrypted access tokens.

How to test

The usage of TLS is machine testable. Follow the latest NCSC guidelines on what is required to test. The serverside is what will be tested, only control over the server is assumed for testing. A testing client will be employed to test adherence of the server. Supporting any protocols, algorithms, key sizes, options or ciphers that are deemed insufficient or phased out by NCSC will lead to failure on the

automated test. Both positive and negative scenarios are part of the test: testing that a subset of *Good* and *Sufficient* configurations are supported and configurations deemed *Insufficient* or marked for *Phase out*. A manual exception to the automated test results can be made when configurations designated for *Phase out* are supported; The API provider will have to provide clear documentation regarding the phase out schedule.

Functional

/core/transport/no-sensitive-uris: No sensitive information in URIs

Statement

Do not put any sensitive information in URIs

Rationale

Even when using TLS connections, information in URIs is not secured. URIs can be cached and logged outside of the servers controlled by clients and servers. Any information contained in them should therefore be considered readable by anyone with access to the network (in the case of the internet, the whole world) and *MUST NOT* contain any sensitive information. This includes client secrets used for authentication, privacy sensitive information such as BSNs or any other information which should not be shared.

Be aware that queries (anything after the '?' in a URI) are also part of a URI.

§ 2.9.1 HTTP-level Security

The guidelines and principles defined in this section are client agnostic. When implementing a client agnostic API, one *SHOULD* at least facilitate that multi-purpose generic HTTP-clients like browsers are able to securely interact with the API. When implementing an API for a specific client it may be possible to limit measures as long as it ensures secure access for this specific client. Nevertheless it is advised to review the following security measures, which are mostly inspired by the OWASP REST Security Cheat Sheet.

Even while remaining client agnostic, clients can be classified in four major groups. This is in line with common practice in *The OAuth 2.0 Authorization Framework*. The groups are:

1. Web applications.
2. Native applications.

3. Browser-based applications.
4. System-to-system applications.

This section contains elements that apply to the generic classes of clients listed above. Although not every client implementation has a need for all the specifications referenced below, a client agnostic API *SHOULD* provide these to facilitate any client to implement relevant security controls.

Most specifications referenced in this section are applicable to the first three classes of clients listed above. Security considerations for native applications are provided in *OAuth 2.0 for Native Apps*, much of which can help non-OAuth2 based implementations as well. For browser-based applications a subsection is included with additional details and information. System-to-system (sometimes called machine-to-machine) may have a need for the listed specifications as well. Note that different usage patterns may be applicable in contexts with system-to-system clients, see above under Client Authentication.

Realizations may rely on internal usage of HTTP-Headers. Information for processing requests and responses can be passed between components, that can have security implications. For instance, this is common practice between a reverse proxy or TLS-offloader and an application server. Additional HTTP headers are used in such example to pass an original IP-address or client certificate.

Implementations *MUST* consider filtering both inbound and outbound traffic for HTTP-headers used internally. The primary focus of inbound filtering is to prevent injection of malicious headers on requests. For outbound filtering, the main concern is leaking of information.

Technical

/core/transport/security-headers: Use mandatory security headers in all API responses

Statement

API security headers *MUST* be returned in all server responses to instruct the client to act in a secure manner

Rationale

There are a number of security related headers that can be returned in the HTTP responses to instruct browsers to act in specific ways. However, some of these headers are intended to be used with HTML responses, and as such may provide little or no security benefits on an API that does not return HTML. The following headers *SHOULD* be included in all API responses:

Header	Rationale
`Cache-Control: no-store`	Prevent sensitive information from being cached.

Header	Rationale
`Content-Security-Policy: frame-ancestors 'none'`	To protect against drag-and-drop style clickjacking attacks.
`Content-Type`	To specify the content type of the response. This *SHOULD* be `application/json` for JSON responses.
`Strict-Transport-Security`	To require connections over HTTPS and to protect against spoofed certificates.
`X-Content-Type-Options: nosniff`	To prevent browsers from performing MIME sniffing, and inappropriately interpreting responses as HTML.
`X-Frame-Options: DENY`	To protect against drag-and-drop style clickjacking attacks.
`Access-Control-Allow-Origin`	To relax the 'same origin' policy and allow cross-origin access. See /core/transport/cors for more information.

The headers below are only intended to provide additional security when responses are rendered as HTML. As such, if the API will never return HTML in responses, then these headers may not be necessary. You *SHOULD* include the headers as part of a defense-in-depth approach if there is any uncertainty about the function of the headers, the types of information that the API returns or information it may return in the future.

Header	Rationale
`Content-Security-Policy: default-src 'none'`	The majority of CSP functionality only affects pages rendered as HTML.
`Feature-Policy: 'none'`	Feature policies only affect pages rendered as HTML.
`Referrer-Policy: no-referrer`	Non-HTML responses should not trigger additional requests.

In addition to the above listed HTTP security headers, web- and browser-based applications *SHOULD* apply *Subresource Integrity*. When using third-party hosted contents, e.g. using a Content Delivery Network, this is even more relevant. While this is primarily a client implementation concern, it may affect the API when it is not strictly segregated or for example when shared supporting libraries are offered.

How to test

The presence of the mandatory security headers can be tested in an automated way. A test client makes a call to the API root. The response is tested for the presence of mandatory headers.

Technical

/core/transport/cors: Use CORS to control access

Statement

CORS *MUST* be used to restrict access from other domains for applicable resources

Rationale

Different resources can have different uses, as some resources are publicly available whereas others are restricted to several domains. Modern web browsers use Cross-Origin Resource Sharing (CORS) to minimize the risk associated with cross-site HTTP-requests.

By default browsers only allow 'same origin' access to resources. This means that responses on requests to another `[scheme]://[hostname]:[port]` than the `Origin` request header of the initial request will not be processed by the browser. To enable cross-site requests APIs can return a `Access-Control-Allow-Origin` response header.

An allowlist *SHOULD* be used to determine the validity of different cross-site requests. To do this, check the `Origin` header of the incoming request and check if the domain in this header is on the allowlist. If this is the case, set the incoming `Origin` header in the `Access-Control-Allow-Origin` response header.

> NOTE
> Using a wildcard `*` in the `Access-Control-Allow-Origin` response header is *NOT RECOMMENDED*, because it disables CORS-security measures. However, if the resource has to be accessed by numerous other origins that are not known up front (such as all resources in an open API, or the `openapi.json` as required by /core/publish-openapi), you *MAY* use `*`.

How to test

Tests of this design rule can only be performed when the intended client is known to the tester. A test can be performed when this information is provided by the API provider. Otherwise no conclusive test result can be reached.

§ 2.9.2 Browser-based applications

A specific subclass of clients are browser-based applications, that require the presence of particular security controls to facilitate secure implementation. Clients in this class are also known as *user-agent-based* or *single-page-applications* (SPA). All browser-based applications *SHOULD* follow the best practices specified in OAuth 2.0 for Browser-Based Apps. These applications can be split into three architectural patterns:

- JavaScript applications with a backend; with this class of applications, the backend is the confidential client and should intermediate any interaction, with tokens never ending up in the browser. Effectively, these are not different from regular web-application for this security facet, even though they leverage JavaScript for implementation.
- JavaScript applications that share a domain with the API (resource server); these can leverage cookies marked as HTTP-Only, Secure and SameSite.
- JavaScript applications without a backend; these clients are considered public clients, and are potentially more vulnerable to several types of attacks, including Cross-Site Scripting (XSS), Cross Site Request Forgery (CSRF) and OAuth token theft. In order to support these clients, the Cross-Origin Resource Sharing (CORS) policy mentioned above is critical and *MUST* be supported.

§ 2.9.3 Validate content types

A REST request or response body *SHOULD* match the intended content type in the header. Otherwise this could cause misinterpretation at the consumer/producer side and lead to code injection/execution.

- Requests containing unexpected or missing content type headers *MUST* be rejected with HTTP response status `406 Not Acceptable` or `415 Unsupported Media Type`.
- Accidentally exposing unintended content types *MUST* be avoided by explicitly defining content types e.g. Jersey (Java) `@consumes("application/json"); @produces("application/json")`. This avoids XXE-attack vectors for example.

It is common for REST services to allow multiple response types (e.g. `application/xml` or `application/json`) in which case then the client specifies the preferred order of response types by the Accept header in the request.

- Do not simply copy the `Accept` header to the `Content-type` header of the response.
- Reject the request (ideally with a `406 Not Acceptable` response) if the Accept header does not specifically contain one of the allowable types.

Services (potentially) including script code (e.g. JavaScript) in their responses *MUST* be especially careful to defend against header injection attacks.

- Ensure the intended Content-Type headers are sent in the response, matching the body content, e.g. `application/json` and not `application/javascript`.

§ 2.10 Normative modules

The following modules are normative for all REST API's.

Functional

/core/modules/geospatial: Apply the geospatial module for geospatial data

Statement

The *API Design Rules Module: Geospatial* version 1.0.x *MUST* be applied when providing geospatial data or functionality.

> NOTE
>
> Geospatial data refers to information that is associated with a physical location on Earth, often expressed by its 2D/3D coordinates.

Rationale

The *API Design Rules Module: Geospatial* formalizes as set of rules regarding:

1. How to encode geospatial data in request and response payloads.
2. How resource collections can be filtered by a given bounding box.
3. How to deal with different coordinate systems (CRS).

Functional

/core/modules/signing: Apply the signing module for signing payloads

Statement

The *API Design Rules Module: Signing* version 1.0.x *MUST* be applied when signing payloads.

> NOTE
>
> This rule does not dictate signing. Instead, it only applies in situations where there is a need for assurance of end to end message integrity and authenticity between client application and server application. In those situations, *API Design Rules Module: Signing* specifies how to sign.

Rationale

The *API Design Rules Module: Signing* formalizes as set of rules regarding:

1. How to sign data in request and response payloads.
2. Which header to specify the signature.

Functional

/core/modules/encryption: Apply the encryption module for encrypting payloads

Statement

The *API Design Rules Module: Encryption* version 1.0.x *MUST* be applied when encrypting payloads.

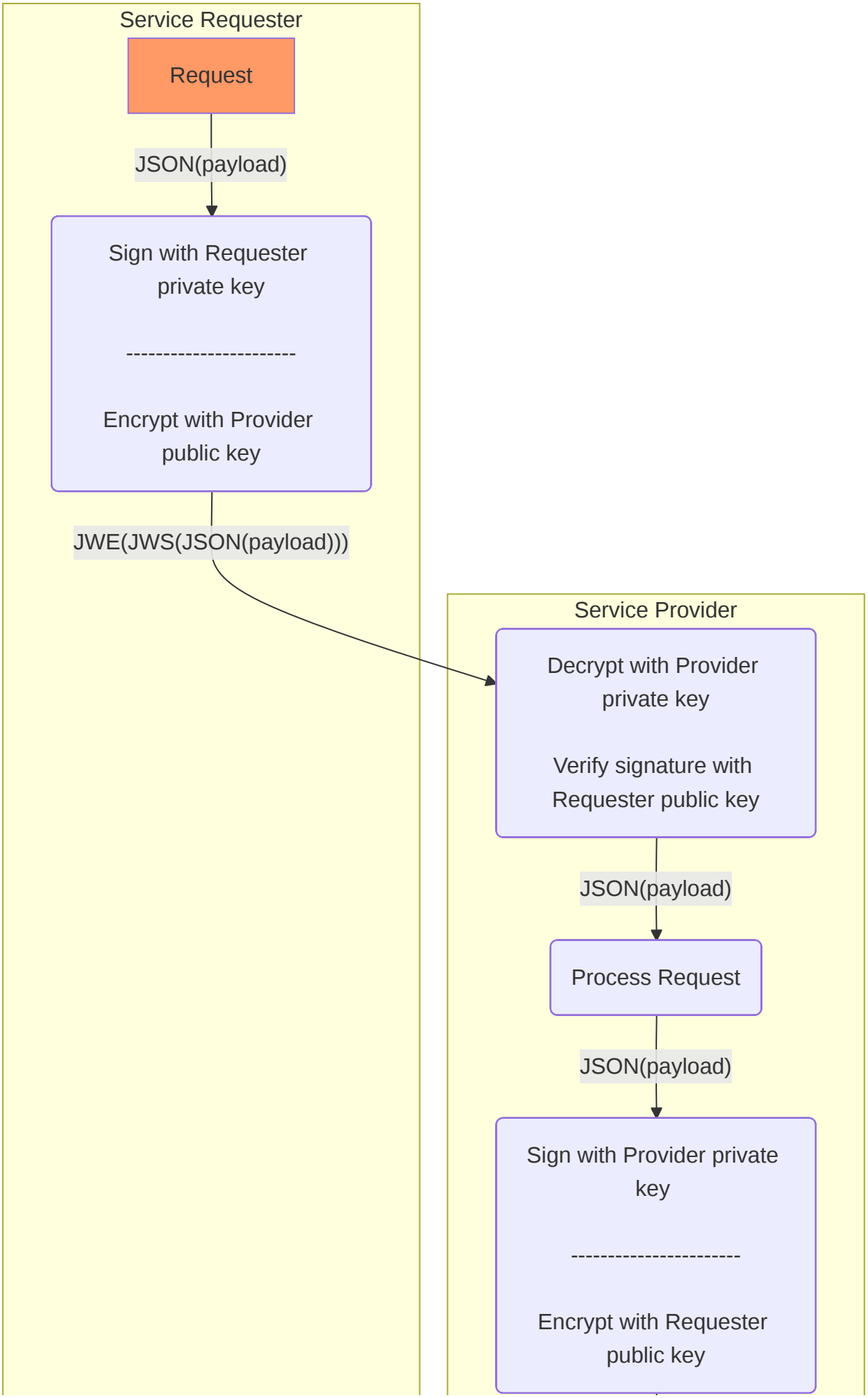
> NOTE
>
> This rule does not dictate encryption. Instead, it only applies in situations where there is a need for end to end message payload confidentiality between client application and server application. In those situations, *API Design Rules Module: Encryption* specifies how to encrypt.

Rationale

The *API Design Rules Module: Encryption* formalizes as set of rules regarding:

1. How to encrypt data in request and response payloads.
2. The flow of operations between client and server.

If both the signing and encryption modules apply, use the following flow of operations:


Service Requester
Request
JSON(payload)
Sign with Requester private key

Encrypt with Provider public key
JWE(JWS(JSON(payload)))
Service Provider
Decrypt with Provider private key
Verify signature with Requester public key
JSON(payload)
Process Request
JSON(payload)
Sign with Provider private key

Encrypt with Requester public key

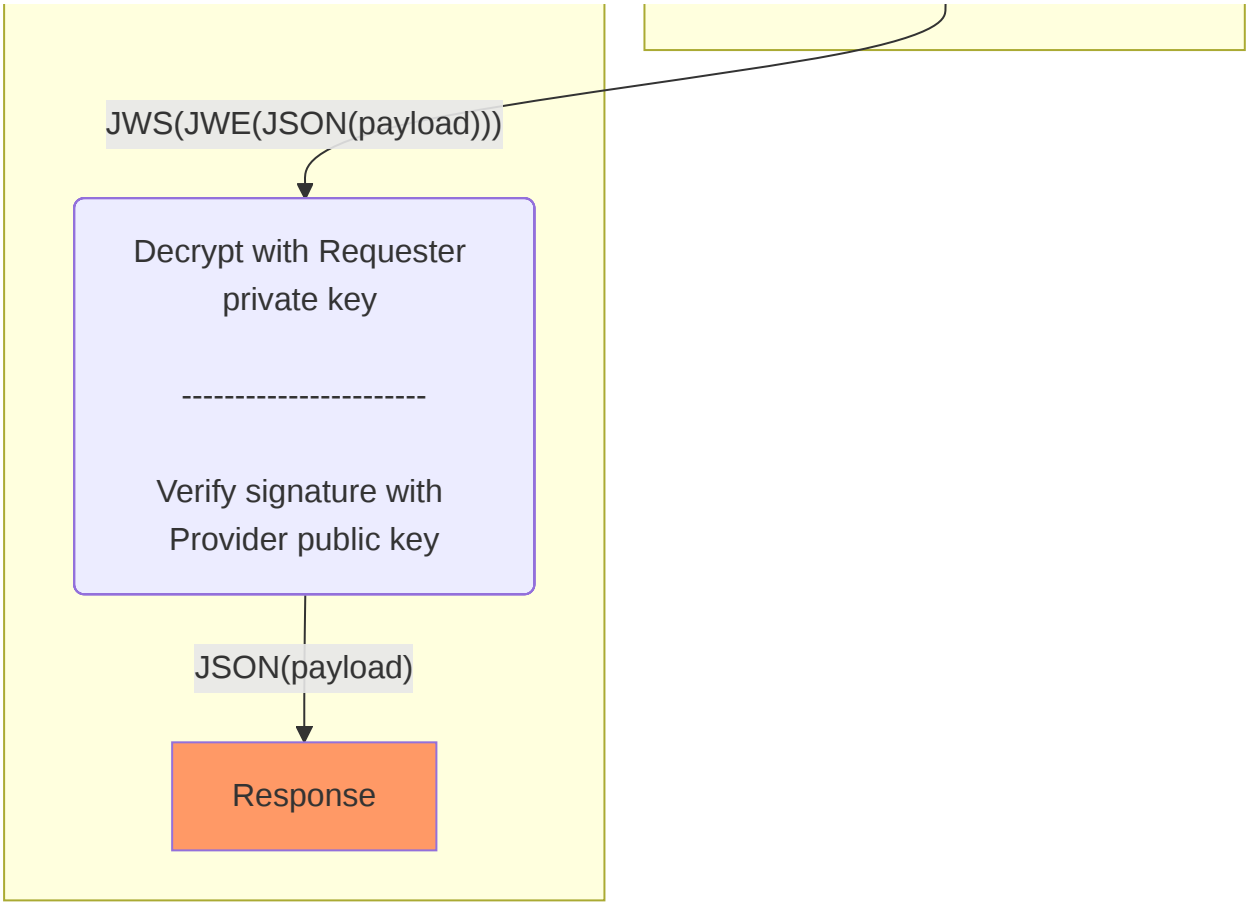


Figure 2 Signing in combination with encryption

§ 3. Glossary

Resource

A resource is an abstraction of a conceptual entity, identified by a globally unique URI, whose current state can be transferred to clients through one or more representations.

Singular resource

A singular resource is a resource that corresponds to a single conceptual entity (e.g. a building, person or event).

Collection resource

A collection resource is a resource that corresponds to a logical grouping of related resources and contains references (URIs) to the individual singular resources it includes. (e.g. a list of buildings).

URI

A URI [*RFC3986*] (Uniform Resource Identifier) is a string that identifies a resource. URIs are intended to be unique across the web, allowing resources to be unambiguously referenced.

OGC

The Open Geospatial Consortium (OGC) is a consortium of experts committed to improving access to geospatial, or location information.

§ A. Linter configuration

This section is non-normative.

▶ Details

Open the above details to inspect the linter configuration. You can also open the configuration file in separate tab

§ B. References

§ B.1 Normative references

[ADR-encryption]
API Design Rules Module: Encryption. P. Haasnoot. Logius. Draft. URL: https://logius-standaarden.github.io/API-mod-encryption/

[ADR-GEO]
API Design Rules Module: Geospatial. L. van den Brink, P. Bresters, P. van Genuchten, G. Mathijssen, M. Strijker. Geonovum. March 07, 2024. URL: https://gitdocumentatie.logius.nl/publicatie/api/mod-geo/

[ADR-signing]
API Design Rules Module: Signing. P. Haasnoot. Logius. Draft. URL: https://logius-standaarden.github.io/API-mod-signing/

[NCSC 2025]
Transport Layer Security (TLS) richtlijnen 2025-05. NCSC. June 2025. URL: https://www.ncsc.nl/wat-kun-je-zelf-doen/documenten/publicaties/2025/juni/01/ict-beveiligingsrichtlijnen-voor-transport-layer-security-2025-05

[OPENAPIS]
OpenAPI Specification. Darrell Miller; Jason Harmon; Jeremy Whitlock; Marsh Gardiner; Mike Ralphson; Ron Ratovsky; Tony Tam; Uri Sarid. OpenAPI Initiative. URL: https://www.openapis.org/

[RFC2119]
Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3986]
Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC8174]
Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[RFC8252]
OAuth 2.0 for Native Apps. W. Denniss; J. Bradley. IETF. October 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8252

[RFC9110]
HTTP Semantics. R. Fielding, Ed.; M. Nottingham, Ed.; J. Reschke, Ed. IETF. June 2022. Internet Standard. URL: https://httpwg.org/specs/rfc9110.html

[SemVer]
Semantic Versioning 2.0.0. T. Preston-Werner. June 2013. URL: https://semver.org

[SRI]
Subresource Integrity. Frederik Braun. W3C. 10 July 2025. W3C Working Draft. URL: https://www.w3.org/TR/sri-2/

§ B.2 Informative references

[OAuth2]
The OAuth 2.0 Authorization Framework. D. Hardt. The Internet Engineering Task Force. October 2012. URL: https://tools.ietf.org/html/rfc6749

↑